

QueryMind AI

Architecture Documentation

Text-to-SQL Analytics Engine

Vijay Mamilla

Codebasics AI Engineering Bootcamp

May 2026

GitHub Repository

github.com/vijaymamilla/querymind

Demo Video

youtube.com/watch?v=lsz7pbXr5Ys

Table of Contents

1. Executive Summary

QueryMind AI is an end-to-end Text-to-SQL Analytics Engine that empowers business users, analysts, and product managers to query relational databases using plain English. Users type a natural language question — such as "What were the top 5 products by revenue last month?" — and QueryMind translates it into valid, executable SQL, runs it against a PostgreSQL 16 database, and returns both structured tabular data and a human-readable natural language summary.

The system eliminates the need for SQL expertise while maintaining strict data governance through configurable guardrails. It is architected as a modular 7-stage pipeline that classifies intent, links schema entities, retrieves semantically similar examples (RAG), generates SQL with GPT-4o, validates syntax and schema, executes safely, and formats the final response.

1.1 Key Performance Metrics

Metric	Value	Details
Execution Accuracy	100%	25 ground-truth queries
Query Coverage	25 queries	4 query types
Average Latency	~3 seconds	End-to-end pipeline
Few-Shot Examples	30 curated pairs	Stored in Qdrant vector DB
Vector Dimensions	1536 dims	text-embedding-3-small

1.2 Why It Matters

Traditional business intelligence tools require SQL knowledge or rely on rigid drag-and-drop interfaces. QueryMind bridges the gap between raw data and business insight by combining:

- Large Language Models (GPT-4o) for intelligent SQL generation
- Retrieval-Augmented Generation (RAG) for few-shot accuracy improvement
- Structural validation (sqlglot AST) that prevents malformed queries
- Runtime guardrails that enforce data access policies without code changes
- Self-healing execution that automatically retries failed queries with error context

QueryMind was developed as part of the Codebasics AI Engineering Bootcamp, demonstrating a production-grade RAG pipeline applied to the domain of database querying. The system achieves 100% execution accuracy across all tested query types — meaning every generated query runs successfully against the database.

2. System Overview

QueryMind AI follows a clean layered architecture with three primary zones: the User Interface (Next.js 14 frontend), the AI Pipeline (FastAPI backend with 7 specialised stages), and the Data & Knowledge Layer (PostgreSQL 16 + Qdrant vector database).

2.1 High-Level Architecture

At the highest level, the system operates as a request-response pipeline:

Zone	Input	Output
User Interface	Natural language question typed by user	Structured JSON result + NL summary
AI Pipeline	Raw question string via REST API	QueryResult: table, columns, rows, summary
Data Layer	Validated SQL statement	PostgreSQL rows + metadata
Vector Layer	Embedded question vector (1536 dims)	Top-4 similar Q-SQL example pairs

2.2 Component Layers

The architecture is divided into four horizontal layers:

Layer	Technologies	Responsibility
Presentation	Next.js 14, React, Tailwind CSS	User interface, query submission, result display
API & Pipeline	FastAPI, Python 3.11+, Pydantic	7-stage NL-to-SQL transformation pipeline
AI & Embeddings	OpenAI GPT-4o, text-embedding-3-small	Classification, generation, embedding, summarisation
Data & Knowledge	PostgreSQL 16, Qdrant, Docling	Schema storage, vector search, document ingestion

2.3 Key Design Principles

Modularity

Each of the 7 pipeline stages is implemented as an independent Python class with a well-defined interface (input schema, output schema). Stages can be tested, replaced, or enhanced in isolation without affecting the rest of the pipeline. This allows, for example, switching from GPT-4o to another LLM by only modifying the Generator stage.

Guardrails by Design

Security and data governance are not afterthoughts — they are enforced at the Validator stage (Stage 5) and are controlled through a database-backed configuration table. Guardrails cannot be bypassed via prompt injection because they are applied after SQL generation, against the parsed AST. Operations such as INSERT, UPDATE, and DELETE can be individually toggled on or off at runtime without restarting any service.

Self-Healing Execution

When the Executor (Stage 6) encounters a SQL runtime error, it does not simply fail. Instead, it constructs a repair prompt that includes the original SQL, the PostgreSQL error message, and the database schema, then re-invokes GPT-4o for a single corrective attempt. This mechanism improves resilience against edge cases in complex multi-join queries.

RAG-Enhanced Generation

Rather than relying solely on GPT-4o's general SQL knowledge, QueryMind augments the generation prompt with the top-4 semantically similar query examples retrieved from Qdrant. This few-shot approach dramatically improves SQL quality for domain-specific patterns such as the e-commerce schema's naming conventions.

Auditability

Every query — whether successful, failed, or blocked — is written to the `query_logs` table with full context: the original question, generated SQL, execution status, error details (if any), row count, and timestamps. This provides a complete audit trail for compliance and debugging.

3. The 7-Stage Pipeline

The core of QueryMind AI is a sequential 7-stage pipeline. Each stage receives a well-defined input, performs a specialised transformation, and passes its output to the next stage. The pipeline is orchestrated by the `QueryPipeline` class in the FastAPI backend. The following diagram shows the data flow:

#	Stage	Input	Output
1	Classifier	Raw user question	query_type + confidence
2	Schema Linker	Question + query_type	Linked schema entities
3	Retriever (RAG)	Question string	Top-4 FewShotPair objects
4	Generator	Schema + examples + question	Raw SQL string
5	Validator	Raw SQL string	Validated SQL or error
6	Executor	Validated SQL	Rows + row_count
7	Formatter	Raw result rows	Structured QueryResult JSON

3.1 Stage 1 — Classifier

The Classifier is the entry point of the pipeline. Its sole responsibility is to determine the intent and structural type of the incoming natural language question before any SQL generation occurs.

Purpose

Classifying the query type upfront allows downstream stages — especially the Retriever — to fetch the most relevant few-shot examples. A simple count query needs different examples than a complex temporal trend query.

Technology

OpenAI GPT-4o via the Chat Completions API with a structured JSON output schema. The model is prompted with the question and a list of valid classes, and it returns a JSON object with `query_type` and `confidence` fields.

Query Types

Query Type	Description	Example Question
SELECT_SIMPLE	Direct lookups with filters, no aggregation or joins	List all customers in New York
SELECT_AGGREGATE	Aggregation functions: COUNT, SUM, AVG, MIN, MAX	Total revenue by product category
SELECT_JOIN	Queries requiring joins across 2+ tables	Top 5 customers by total order value
SELECT_TEMPORAL	Time-based filtering, trends, date arithmetic	Monthly revenue for the past 6 months

BLOCKED	Disallowed intent (DDL, DML mutations, schema dumps)	Delete all orders from 2023
----------------	--	-----------------------------

Input / Output

Direction	Value
Input	Raw user question string
Output	JSON: { query_type: string, confidence: float (0-1), reasoning: string }

Key Implementation Details

- Uses GPT-4o with response_format: { type: 'json_object' } for reliable structured output
- Confidence score enables downstream alerting for low-confidence classifications
- BLOCKED type is returned for queries that match disallowed patterns (e.g., requests to drop tables, reveal schema secrets)
- The reasoning field provides a human-readable explanation of why the type was assigned

3.2 Stage 2 — Schema Linker

The Schema Linker bridges the gap between natural language vocabulary and actual database schema names. Users naturally use business terms that do not match database column names. The Schema Linker resolves these ambiguities before the Generator attempts to write SQL.

Purpose

Without schema linking, GPT-4o would need to guess column names and might hallucinate table or column names that do not exist. The Schema Linker provides authoritative mappings so the Generator receives a schema context grounded in the real database structure.

Technology

Live PostgreSQL introspection combined with the `schema_columns` admin table, which stores column synonyms and descriptions. The linker uses both fuzzy string matching and GPT-4o reasoning to map business terms to schema entities.

Example Mappings

User Term	Maps To	Explanation
revenue	orders.total_amount	Business term for the monetary value stored in total_amount column
customer name	first_name + last_name	Name is split across two columns in the customers table
product price	products.unit_price	Business concept maps to the specific column name in schema
order date	orders.order_date	Direct mapping with minor vocabulary difference
category	categories.category_name	Short form mapped to full qualified table.column reference

Input / Output

Direction	Value
Input	Question string + query_type
Output	SchemaContext: { tables: string[], columns: { [table]: string[] }, join_hints: string[], entity_mappings: object }

Key Implementation Details

- Performs live `INFORMATION_SCHEMA` queries to get current table and column lists
- Consults `schema_columns` admin table for synonyms (e.g., 'revenue' is a synonym for 'total_amount')
- Uses `query_type` to prioritise which tables to include in the schema context
- Produces `join_hints` when the question references entities from multiple tables
- Limits the schema context to relevant tables to avoid overwhelming the Generator prompt

3.3 Stage 3 — Retriever (RAG)

The Retriever implements Retrieval-Augmented Generation (RAG) to find the most semantically relevant few-shot examples for the current question. Instead of relying on GPT-4o's general SQL knowledge, the system provides concrete examples from the same domain and schema, significantly improving generation accuracy.

Purpose

Few-shot examples teach GPT-4o the specific patterns, table names, join structures, and formatting conventions of the QueryMind e-commerce schema. Without retrieval, the Generator would need to infer all schema details from the system prompt alone.

Technology

OpenAI text-embedding-3-small model for question embedding (1536 dimensions), Qdrant vector database for similarity search, AsyncQdrantClient for non-blocking I/O.

How It Works

1. The user question is embedded using OpenAI text-embedding-3-small, producing a 1536-dimensional float vector
2. The vector is sent to Qdrant's `query_points()` API against the `few_shot_examples` collection
3. Qdrant performs approximate nearest-neighbour search using cosine similarity
4. The top-4 results (question + SQL pairs) are returned as `FewShotPair` objects
5. These examples are passed to the Generator to be included in the prompt

Qdrant Collection: `few_shot_examples`

Property	<code>few_shot_examples</code>	<code>schema_docs</code>
Content	30 Q-to-SQL pairs	Document chunks from uploaded files
Dimensions	1536	1536
Distance Metric	Cosine	Cosine
Indexed Field	question (text)	chunk_text (text)
Payload Fields	question, sql, query_type	source_file, chunk_index, word_count

Input / Output

Direction	Value
Input	Question string
Output	List[FewShotPair]: [{ question: string, sql: string, similarity_score: float }]

Key Implementation Details

- 30 curated Q-to-SQL examples span all 4 query types (7-8 per type)
- Cosine similarity chosen over Euclidean distance for normalised embedding vectors

- Top-4 retrieved (not top-1) to provide diversity in few-shot patterns
- Async retrieval does not block the main thread, keeping pipeline latency low
- New examples can be added via the Admin Examples Manager without restarting services

3.4 Stage 4 — Generator

The Generator is the heart of the SQL generation process. It assembles a structured prompt from all upstream context and invokes GPT-4o to produce the SQL statement.

Purpose

The Generator translates the enriched context (schema entities, few-shot examples, guardrail hints) into an effective prompt and extracts the clean SQL string from GPT-4o's response.

Technology

OpenAI GPT-4o via the Chat Completions API. The model is configured with a low temperature (0.1-0.2) for deterministic, factual SQL generation rather than creative output.

Prompt Structure

The Generator builds a three-part prompt:

- System message: Role definition (expert PostgreSQL developer), schema context, guardrail constraints, output format instructions
- Few-shot examples: Top-4 retrieved Q-to-SQL pairs formatted as user/assistant message pairs
- User message: The actual question with linked schema entities highlighted

Example prompt construction:

```
SYSTEM:
You are an expert PostgreSQL developer. Generate only valid SQL.
Schema: customers(id, first_name, last_name, email, city, created_at)
        orders(id, customer_id, total_amount, status, order_date)
Rules: SELECT only. LIMIT 500 rows maximum.

EXAMPLE 1:
User: List all customers in New York
Assistant: SELECT first_name, last_name, email FROM customers
           WHERE city = 'New York' LIMIT 500;

USER:
Show me the top 5 customers by total order value
```

Input / Output

Direction	Value
Input	SchemaContext + List[FewShotPair] + question string
Output	Raw SQL string (may include prefix like 'SQL:' which is stripped)

Key Implementation Details

- Response cleaning: strips 'SQL:', '```sql', '```' prefixes and trailing whitespace
- GPT-4o temperature set to 0.1 for deterministic SQL output
- Schema context is formatted as compact CREATE TABLE-style DDL for token efficiency
- Guardrail hints are injected into the system prompt (e.g., 'Do not generate INSERT/UPDATE/DELETE')
- Output is a raw SQL string — not yet validated or executed

3.5 Stage 5 — Validator

The Validator is the safety gate of the pipeline. Every SQL string generated by GPT-4o must pass through the Validator before it can be executed. The Validator performs three types of checks: syntax validation, schema validation, and guardrail enforcement.

Purpose

Even with few-shot examples and schema context, GPT-4o can occasionally generate SQL with syntax errors, reference non-existent tables, or attempt disallowed operations. The Validator catches all of these before any database interaction occurs.

Technology

sqlglot — a Python SQL parsing library that supports 20+ SQL dialects including PostgreSQL. Validation is performed against the Abstract Syntax Tree (AST), which is more reliable and precise than regular expression matching.

Validation Steps

6. Syntax Parsing: `sqlglot.parse_one(sql, dialect='postgres')` — raises `ParseError` for malformed SQL
7. Statement Type Check: Extracts the root statement type from the AST (SELECT, INSERT, UPDATE, DELETE)
8. Guardrail Check: Compares the statement type against the `guardrail_config` table settings
9. Table Validation: Extracts all referenced table names from the AST and checks them against the known schema
10. Column Spot-Check: Validates that referenced columns exist in the corresponding tables

Check	Method	Failure Handling
SQL Syntax	sqlglot AST parse	Returns validation error; SQL not executed
Statement Type	AST root node type	Compares against <code>guardrail_config</code> flags
Guardrail Rules	DB lookup on <code>guardrail_config</code>	Returns 403-style error with reason
Table Names	AST table extraction	Flags unknown tables; soft warning

Input / Output

Direction	Value
Input	Raw SQL string
Output	<code>ValidationResult: { is_valid: bool, sql: string, error: string null, statement_type: string }</code>

Key Implementation Details

- Uses AST table extraction — not regex — to prevent false positives from table names in string literals
- Guardrail settings are loaded from the database at request time (not cached), enabling real-time changes
- A blocked statement type results in a 403-equivalent error with an informative message

- Validation errors are logged to query_logs with the invalid SQL preserved for debugging

3.6 Stage 6 — Executor

The Executor runs the validated SQL against the PostgreSQL 16 database and returns the result rows. It also implements the self-healing retry mechanism for handling runtime SQL errors.

Purpose

The Executor bridges the AI pipeline and the actual database. It handles connection pooling, row limiting, error capture, and the self-healing retry loop — all in a way that is transparent to the user.

Technology

asyncpg (async PostgreSQL driver for Python), combined with a connection pool managed by FastAPI's lifespan context. The self-healing component re-invokes GPT-4o via the same API client as the Generator.

Execution Flow

11. Receives validated SQL from the Validator
12. Appends LIMIT {max_rows} if no LIMIT clause is present in the SQL
13. Opens a connection from the pool and executes the query
14. If successful: returns rows and row_count to the Formatter
15. If PostgreSQL error: captures the error message and initiates self-healing retry (see Section 9)
16. If retry succeeds: returns the corrected query result
17. If retry fails: returns error to the caller with full context

Input / Output

Direction	Value
Input	Validated SQL string + max_rows limit
Output	ExecutionResult: { rows: List[dict], row_count: int, sql_executed: string, error: string null }

Key Implementation Details

- max_rows defaults to 500 and is configurable in guardrail_config table
- LIMIT is applied programmatically if the generated SQL does not already include one
- Self-healing is limited to one retry to prevent infinite loops
- Row results are serialised to List[dict] using asyncpg's built-in Record-to-dict conversion
- Connection pool size is configurable via environment variable POOL_SIZE (default: 10)

3.7 Stage 7 — Formatter

The Formatter is the final stage of the pipeline. It takes the raw execution results and transforms them into the structured QueryResult JSON that the frontend consumes, adding a natural language summary generated by GPT-4o.

Purpose

Raw database rows are not user-friendly on their own. The Formatter structures the data for table rendering (columns array, rows array), computes metadata (row_count), and generates a one-paragraph natural language summary that explains the results in business terms.

Technology

OpenAI GPT-4o for natural language summary generation. The summary prompt includes the original question, the row count, and a sample of the result data (first 10 rows) to keep token usage bounded.

Output Schema — QueryResult

Field	Type	Description
table	List[dict]	Each dict is one result row with column names as keys
columns	List[str]	Ordered list of column names for the result table header
row_count	int	Total number of rows returned (may be capped at max_rows)
nl_summary	str	GPT-4o generated natural language interpretation of results
sql_executed	str	The final SQL that was executed (after any self-healing corrections)
query_type	str	The classified query type from Stage 1
latency_ms	int	End-to-end pipeline latency in milliseconds

Natural Language Summary Example

For a query that returns the top 5 customers by revenue, GPT-4o might generate:

```
"The top 5 customers by total order value are: John Smith ($12,450), Maria Garcia ($10,200), David Chen ($9,875), Sarah Johnson ($8,640), and Michael Brown ($7,920). Together, these customers account for approximately $49,085 in total revenue."
```

Input / Output

Direction	Value
Input	ExecutionResult (rows + metadata) + original question
Output	QueryResult: { table: List[dict], columns: List[str], row_count: int, nl_summary: str, sql_executed: str }

Key Implementation Details

- GPT-4o summary generation uses temperature 0.3 for slightly more natural phrasing
- Only the first 10 rows are sent to GPT-4o for summary to minimise token cost
- Column names are formatted for readability (snake_case to Title Case)
- The `sql_executed` field passes through the final SQL (post self-healing if applicable)

4. Data & Knowledge Layer

The Data & Knowledge Layer comprises all persistent storage components that the pipeline reads from and writes to. This includes the operational PostgreSQL database, the Qdrant vector store, and the admin database tables.

4.1 PostgreSQL 16 — E-commerce Schema

QueryMind operates against a synthetic but realistic e-commerce database containing 7 interrelated tables. The schema is seeded with 1000+ synthetic rows per table to provide meaningful query results during development and evaluation.

Table	Primary Key	Row Count	Key Relationships
customers	id (SERIAL)	1000+	Referenced by orders, reviews
orders	id (SERIAL)	1000+	FK: customer_id → customers
order_items	id (SERIAL)	3000+	FK: order_id → orders, product_id → products
products	id (SERIAL)	200+	FK: category_id, supplier_id
categories	id (SERIAL)	20+	Self-referencing parent_category_id
suppliers	id (SERIAL)	50+	Referenced by products
reviews	id (SERIAL)	500+	FK: product_id, customer_id

Customers Table

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique customer identifier
first_name	VARCHAR(100)	Customer first name
last_name	VARCHAR(100)	Customer last name
email	VARCHAR(200) UNIQUE	Customer email address
city	VARCHAR(100)	City of residence
country	VARCHAR(100)	Country of residence
created_at	TIMESTAMP	Account creation timestamp

Orders Table

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique order identifier

customer_id	INTEGER FK	References customers.id
total_amount	NUMERIC(10,2)	Total monetary value of the order
status	VARCHAR(50)	Order status: pending, shipped, delivered, cancelled
order_date	DATE	Date the order was placed
shipping_addresses	TEXT	Full delivery address

Order Items Table

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique item identifier
order_id	INTEGER FK	References orders.id
product_id	INTEGER FK	References products.id
quantity	INTEGER	Number of units ordered
unit_price	NUMERIC(10,2)	Price per unit at time of order
subtotal	NUMERIC(10,2)	quantity × unit_price

Products Table

Column	Type	Description
id	SERIAL PRIMARY KEY	Unique product identifier
product_name	VARCHAR(200)	Product display name
category_id	INTEGER FK	References categories.id
supplier_id	INTEGER FK	References suppliers.id
unit_price	NUMERIC(10,2)	Current selling price
stock_quantity	INTEGER	Current inventory level
created_at	TIMESTAMP	Product listing timestamp

Other Tables

- categories: id, category_name, description, parent_category_id
- suppliers: id, company_name, contact_name, email, country, phone
- reviews: id, product_id, customer_id, rating (1-5), review_text, created_at

4.2 Qdrant Vector Database

Qdrant is an open-source vector similarity search engine optimised for high-performance ANN (Approximate Nearest Neighbour) search. QueryMind uses Qdrant to power the RAG retrieval in Stage 3.

Qdrant runs as a Docker container alongside the FastAPI backend. The AsyncQdrantClient is initialised at application startup and reused for all vector queries, eliminating connection overhead per request.

Collections Overview

Property	few_shot_examples	schema_docs
Purpose	Store curated Q-to-SQL examples for RAG retrieval	Store chunked domain documentation for schema context retrieval
Vector Dimensions	1536	1536
Distance Metric	Cosine	Cosine
Payload	question (str), sql (str), query_type (str)	chunk_text (str), source_file (str), chunk_index (int)
Current Size	30 vectors	Variable (depends on ingested docs)
Top-K Retrieval	4 results	5 results

Vector Search Configuration

- Embedding model: text-embedding-3-small (OpenAI)
- Dimensions: 1536 floats per vector
- Distance metric: Cosine similarity
- Indexing: HNSW (Hierarchical Navigable Small World) for efficient ANN
- Top-K retrieval: 4 results for few_shot_examples, 5 results for schema_docs

4.3 Docling Document Ingestion

Docling is an open-source document parsing library that enables QueryMind to ingest structured documents (technical documentation, data dictionaries, schema descriptions) and index them into the schema_docs Qdrant collection.

Supported Formats

- PDF (including multi-column and table-heavy documents)
- DOCX (Microsoft Word documents)
- HTML (web pages and documentation sites)
- Markdown (README files, wiki pages)

Chunking Strategy

Parameter	Value / Description
Chunk size	400 words per chunk

Chunk overlap	50 words overlap between consecutive chunks
Chunking method	Word-count based sliding window
Deduplication	Deterministic UUID from SHA-256 hash of chunk text
Target collection	schema_docs in Qdrant

Deduplication

Each chunk is assigned a deterministic UUID based on the hash of its text content. When a document is re-ingested, existing chunks with the same hash are skipped, preventing duplicate vectors in Qdrant.

4.4 Admin Database Tables

In addition to the operational e-commerce tables, QueryMind maintains a set of admin tables in the same PostgreSQL database. These tables power the Admin Panel and store operational configuration.

Table	Purpose	Key Columns
few_shot_examples	Persistent store for Q-to-SQL pairs; synced to Qdrant on save	id, question, sql, query_type, created_at
schema_columns	Column descriptions and business term synonyms for schema linking	table_name, column_name, description, synonyms (JSONB)
guardrail_config	Runtime toggles for SQL operation permissions and row limits	key, value, description, updated_at
model_config	Active LLM model name, SQL dialect, and inference parameters	key, value, updated_at
query_logs	Full audit trail of every query processed by the system	id, question, sql, status, row_count, error, latency_ms, created_at

5. Admin Panel

The Admin Panel is a Next.js 14 web interface that provides system operators and administrators with full control over QueryMind's runtime behaviour. It is accessible at the `/admin` route and requires authentication.

The panel is divided into five specialised managers, each corresponding to a core operational concern:

Admin Screen	URL	Primary Function
Schema Manager	<code>/admin/schema</code>	Manage column descriptions and business term synonyms
Examples Manager	<code>/admin/examples</code>	Curate few-shot Q-to-SQL pairs for RAG retrieval
Query Logs	<code>/admin/logs</code>	Full audit trail with filtering and CSV export
Guardrails Config	<code>/admin/guardrails</code>	Runtime SQL operation permission toggles and row limits
Model Config	<code>/admin/config</code>	LLM model selection, inference parameters, SQL dialect

5.1 Schema Manager

URL: `/admin/schema`

The Schema Manager provides a visual interface for managing the column metadata and synonym mappings that power Stage 2 (Schema Linker). Operators can view all tables and columns in the connected PostgreSQL database and enrich them with business descriptions and synonyms.

Key Features

- Browse all tables and columns in the connected database
- Add descriptions to columns (e.g., `'total_amount: The total monetary value of this order in USD'`)
- Define synonyms for columns (e.g., synonyms for `'total_amount'`: `[revenue, sales, income]`)
- Bulk import column metadata from CSV or JSON
- Live preview of how schema context will appear in the Generator prompt

5.2 Examples Manager

URL: `/admin/examples`

The Examples Manager is the interface for curating the few-shot Q-to-SQL examples stored in Qdrant's `few_shot_examples` collection. High-quality examples directly improve SQL generation accuracy.

Key Features

- View all 30 curated examples with their query types and SQL
- Add new Q-to-SQL pairs with automatic Qdrant vector indexing
- Edit existing examples (updates both PostgreSQL and Qdrant)
- Delete examples with automatic Qdrant vector removal

- Filter examples by query type (SELECT_SIMPLE, SELECT_AGGREGATE, etc.)
- Test retrieval: enter a question and see which examples would be retrieved

5.3 Query Logs

URL: /admin/logs

The Query Logs viewer provides a full audit trail of every query submitted to QueryMind. This is essential for monitoring system health, debugging failures, and analysing usage patterns.

Key Features

- Paginated table of all query executions with timestamps
- Filter by status (success / error / blocked), query type, and date range
- Click any row to see full details: question, generated SQL, error message, execution time
- Export logs to CSV for offline analysis
- Automatic real-time updates (polling every 30 seconds)

5.4 Guardrails Configuration

URL: /admin/guardrails

The Guardrails Config page allows operators to modify the safety settings of the system at runtime. Changes take effect immediately for new queries — no server restart required.

Key Features

- Toggle allow_select, allow_insert, allow_update, allow_delete individually
- Set the max_rows limit (default: 500)
- View the current effective configuration at a glance
- Configuration change history with timestamps

5.5 Model Configuration

URL: /admin/config

The Model Config page controls which LLM model and SQL dialect are active. This enables operators to switch between model versions (e.g., gpt-4o vs gpt-4o-mini) to balance cost and quality.

Key Features

- Select the active model from a dropdown of configured options
- Adjust inference parameters: temperature, max_tokens, top_p
- Switch SQL dialect (postgres, mysql, sqlite) for multi-database deployments
- View token usage statistics for cost monitoring

6. Guardrails & Safety

QueryMind implements a multi-layer safety system designed to prevent data modification, enforce access policies, and protect against prompt injection attacks. The guardrail system is a first-class architectural component, not an afterthought.

6.1 Guardrail Architecture

Guardrails are enforced at two layers: the Classifier (Stage 1) provides a first-pass BLOCKED classification for obviously disallowed intents, and the Validator (Stage 5) performs the authoritative enforcement using AST-based SQL parsing against the `guardrail_config` table.

The key security property is that guardrails are applied post-generation. Even if a user crafts a prompt that tricks GPT-4o into generating a DELETE statement, the Validator will catch and reject it before it reaches the Executor. Prompt injection cannot bypass guardrails because the validation happens outside the LLM's context.

6.2 Guardrail Settings

Guardrail Key	Default Value	Type	Description
allow_select	true	boolean	Permit SELECT queries (should generally always be true)
allow_insert	false	boolean	Permit INSERT statements (disabled by default for safety)
allow_update	false	boolean	Permit UPDATE statements (disabled by default for safety)
allow_delete	false	boolean	Permit DELETE statements (disabled by default for safety)
max_rows	500	integer	Maximum rows returned per query (applied as LIMIT)

6.3 Runtime Configurability

All guardrail settings are stored in the `guardrail_config` PostgreSQL table. The Validator reads these settings at request time (not cached at startup), which means changes made via the Admin Panel take effect for the next query without any service restart.

The `guardrail_config` table structure:

```
CREATE TABLE guardrail_config (
  key          VARCHAR(100) PRIMARY KEY,
  value        VARCHAR(200) NOT NULL,
  description  TEXT,
  updated_at   TIMESTAMP DEFAULT NOW()
);
```

6.4 Blocked Query Handling

When a query is blocked by a guardrail, the system:

- Returns a 400 Bad Request response with a clear, user-friendly error message
- Logs the blocked query to `query_logs` with status = 'blocked' and the matching guardrail key
- Does NOT execute any database operation
- Does NOT expose internal schema information in the error message

6.5 Prompt Injection Resistance

QueryMind's architecture is resistant to prompt injection by design. The key properties are:

- SQL guardrails are enforced at the AST level, not by checking the prompt text
- The user's input is treated as data (to be embedded and classified), never as instructions
- Schema context is injected by the system, not by the user
- The Classifier's BLOCKED type provides an early rejection for obvious attack patterns

7. Self-Healing SQL

Self-healing SQL is a resilience mechanism in Stage 6 (Executor) that automatically attempts to correct SQL errors at runtime. When GPT-4o generates syntactically valid but semantically incorrect SQL (e.g., wrong column references, incompatible type comparisons), the self-healing loop provides a second attempt with the error context included.

7.1 When Self-Healing Triggers

Self-healing is activated when:

- The SQL passes Validator checks (syntax and schema validation succeed)
- PostgreSQL returns a runtime error during execution
- This is the first execution attempt (not already a retry)

Self-healing does NOT trigger for:

- Validation failures (the SQL never reaches the Executor)
- Guardrail blocks
- Second-attempt failures (no infinite retry loop)

7.2 Self-Healing Flow

18. Original SQL is executed against PostgreSQL
19. PostgreSQL returns an error (e.g., 'column does not exist', 'invalid input syntax')
20. Executor captures the full error message and the failed SQL
21. A repair prompt is constructed with: original question, failed SQL, PostgreSQL error, and full schema context
22. GPT-4o is invoked with the repair prompt at temperature 0.0 (maximally deterministic)
23. The corrected SQL is extracted and re-validated through the Validator
24. The corrected SQL is executed — if it succeeds, the result is returned normally
25. If the corrected SQL also fails, the error is returned to the caller with full context

7.3 Repair Prompt Structure

```
SYSTEM: You are a PostgreSQL expert. Fix the SQL error below.

ORIGINAL QUESTION: [user's question]

FAILED SQL:
SELECT customer_name FROM orders WHERE total > 1000;

POSTGRES SQL ERROR:
column 'customer_name' does not exist.
HINT: column is 'first_name' in table 'customers'

SCHEMA CONTEXT:
customers(id, first_name, last_name, email, city)
orders(id, customer_id, total_amount, status, order_date)

Return only corrected SQL, no explanation.
```

7.4 Performance

In the QueryMind evaluation suite (25 queries), self-healing was required for approximately 2-3 queries per run. All triggered self-healing attempts succeeded, contributing to the overall 100% execution accuracy metric. The self-healing retry adds approximately 1-2 additional seconds to the end-to-end latency for affected queries.

8. Evaluation Results

QueryMind includes a formal evaluation harness that measures the system's accuracy against a curated set of ground-truth query pairs. This provides objective performance measurement beyond anecdotal testing.

8.1 Evaluation Harness

File: eval/run_eval.py

The evaluation harness reads a JSON file of ground-truth pairs (question → expected_sql), submits each question to the live QueryMind pipeline, and compares the result. Two metrics are measured for each query:

- Execution Accuracy: Did the generated SQL execute without errors and return rows?
- Exact Match: Does the generated SQL exactly match the ground-truth SQL (after normalisation)?

8.2 Results Summary

Query Type	Count	Execution Accuracy	Exact Match
SELECT_SIMPLE	6	100%	16.7% (1/6)
SELECT_AGGREGATE	7	100%	0% (0/7)
SELECT_JOIN	7	100%	14.3% (1/7)
SELECT_TEMPORAL	5	100%	0% (0/5)

8.3 Analysis

Execution Accuracy — 100%

All 25 test queries executed successfully without runtime errors. This is the primary production metric — it means QueryMind never returns a broken query to the user. The combination of few-shot RAG retrieval, schema linking, validation, and self-healing ensures that every query runs.

Exact Match Rate — Low by Design

The exact match rate is intentionally low and does not indicate poor performance. There are two key reasons:

- LIMIT 500: The max_rows guardrail appends LIMIT 500 to all queries. Ground-truth SQL may not include this clause, causing an exact mismatch even when the logical query is identical.
- SQL Equivalence: Multiple semantically equivalent SQL statements can answer the same question. For example, a JOIN using explicit syntax vs. implicit syntax, or different column ordering in SELECT, would fail exact match while being functionally correct.

A more appropriate metric for production evaluation would be result-set accuracy: do the returned rows match the expected rows? Future evaluation work would incorporate this metric.

8.4 Running the Evaluation

```
# From the project root
cd eval
python run_eval.py --dataset ground_truth.json --output results.json

# View summary
python run_eval.py --summary results.json
```

9. Technology Stack

QueryMind integrates a modern, production-grade technology stack spanning AI/ML, backend services, frontend, and infrastructure layers. The following table provides a complete reference.

Layer	Technology	Version	Purpose
AI / LLM	OpenAI GPT-4o	Latest	Classification, SQL generation, NL summary, self-healing
AI / LLM	text-embedding-3-small	Latest	Question and example embedding for vector search
Vector DB	Qdrant	1.9+	Few-shot example and schema document vector storage and retrieval
Backend	FastAPI	0.110+	REST API framework, async request handling, OpenAPI docs
Backend	Python	3.11+	Pipeline implementation language
Backend	asyncpg	0.29+	Async PostgreSQL driver with connection pooling
Backend	Pydantic	v2	Data validation and schema definition for API models
SQL	sqlglot	Latest	SQL parsing, AST extraction, syntax validation (20+ dialects)
Database	PostgreSQL	16	Primary operational database: e-commerce schema + admin tables
Frontend	Next.js	14	React-based web UI with App Router and SSR
Frontend	TypeScript	5+	Type-safe frontend development
Frontend	Tailwind CSS	3	Utility-first CSS framework for admin and query UI
Ingestion	Docling	Latest	Document parsing and chunking for schema_docs indexing
Infra	Docker Compose	Latest	Orchestrate PostgreSQL and Qdrant containers for local dev

9.1 Technology Selection Rationale

GPT-4o

Selected for its superior reasoning capabilities for structured output generation. The model's ability to generate valid SQL from natural language, understand schema relationships, and reason about query semantics makes it the best choice for this use case. The JSON response format feature ensures reliable structured outputs from the Classifier and Formatter stages.

Qdrant

Chosen over alternatives (Pinecone, Weaviate, pgvector) for its excellent async Python client, Docker-based deployment simplicity, and high performance on small-to-medium vector collections. The payload filtering capability allows efficient retrieval by `query_type`.

FastAPI

Provides automatic OpenAPI documentation, Pydantic model validation for all request/response schemas, and excellent async support for non-blocking database and API calls. The dependency injection system cleanly manages shared resources (database pools, Qdrant clients).

sqlglot

A pure-Python, zero-dependency SQL parser that supports 20+ dialects. Unlike alternatives, sqlglot provides full AST access (not just syntax checking), enabling precise table/column extraction for schema validation without regex fragility.

Next.js 14

Server-side rendering for fast initial page loads, App Router for clean URL structure, and built-in TypeScript support. The streaming API integration allows the frontend to display partial results as they arrive from the pipeline.

10. API Endpoints Reference

QueryMind exposes a RESTful API via FastAPI. All endpoints accept and return JSON. The interactive API documentation is available at <http://localhost:8000/docs> (Swagger UI) when running locally.

10.1 Core Query Endpoints

Method	Endpoint	Description
POST	/api/query	Submit a natural language question; returns full QueryResult
GET	/api/health	Health check: returns status of DB, Qdrant, and OpenAI connectivity
GET	/api/schema	Returns the current database schema (tables and columns)
POST	/api/classify	Classify a question without full pipeline execution (debug)

10.2 Admin Endpoints

Method	Endpoint	Description
GET	/api/admin/examples	List all few-shot examples
POST	/api/admin/examples	Create new example (saves to DB + Qdrant)
PUT	/api/admin/examples/{id}	Update existing example
DELETE	/api/admin/examples/{id}	Delete example from DB and Qdrant
GET	/api/admin/schema-columns	List all schema column metadata
POST	/api/admin/schema-columns	Add or update column description / synonyms
GET	/api/admin/guardrails	Get current guardrail configuration
PUT	/api/admin/guardrails/{key}	Update a guardrail setting by key
GET	/api/admin/logs	Paginated query logs with filter support
GET	/api/admin/model-config	Get active model and inference configuration
PUT	/api/admin/model-config/{key}	Update model configuration value

10.3 Example Request & Response

POST /api/query — Submit a natural language question:

```
# Request
POST /api/query HTTP/1.1
Content-Type: application/json

{
  "question": "What are the top 5 customers by total order value?"
}
```

```
# Response (200 OK)
{
  "query_type": "SELECT_JOIN",
  "sql_executed": "SELECT c.first_name, c.last_name,
    SUM(o.total_amount) AS total_value
  FROM customers c JOIN orders o ON c.id = o.customer_id
  GROUP BY c.id ORDER BY total_value DESC LIMIT 5;",
  "columns": ["first_name", "last_name", "total_value"],
  "table": [{"first_name": "John", "last_name": "Smith", "total_value": 12450.00}, ...],
  "row_count": 5,
  "nl_summary": "The top 5 customers by total order value are...",
  "latency_ms": 2847
}
```


11. Setup & Deployment

This section covers everything needed to run QueryMind AI locally for development or evaluation. The system uses Docker Compose to manage infrastructure services (PostgreSQL and Qdrant), with the FastAPI backend and Next.js frontend running directly on the host machine.

11.1 Prerequisites

Prerequisite	Version	Notes
Python	3.11+	Required for FastAPI backend
Node.js	18+	Required for Next.js frontend
Docker Desktop	Latest	Required to run PostgreSQL and Qdrant containers
Docker Compose	v2+	Included with Docker Desktop
OpenAI API Key	N/A	Must have GPT-4o and embedding model access
Git	Latest	For cloning the repository

11.2 Step-by-Step Setup

Step 1: Clone the Repository

```
git clone https://github.com/vijaymamilla/querymind.git
cd querymind
```

Step 2: Start Infrastructure Services

```
# Start PostgreSQL 16 and Qdrant via Docker Compose
docker compose up -d

# Verify services are running
docker compose ps
# Expected: querymind-postgres (healthy), querymind-qdrant (healthy)
```

Step 3: Configure Backend Environment

```
cd backend
cp .env.example .env

# Edit .env and set your values:
# OPENAI_API_KEY=sk-...
# DATABASE_URL=postgresql://querymind:querymind@localhost:5432/querymind
# QDRANT_URL=http://localhost:6333

# Create virtual environment
python -m venv venv
source venv/bin/activate # Linux/macOS
# OR: venv\Scripts\activate # Windows

# Install Python dependencies
pip install -r requirements.txt
```

Step 4: Seed the Database

```
# Run database migrations
python scripts/migrate.py

# Seed e-commerce tables with synthetic data
python scripts/seed_db.py

# Seed few-shot examples into PostgreSQL + Qdrant
python scripts/seed_examples.py
```

Step 5: Start the FastAPI Backend

```
cd backend
uvicorn main:app --host 0.0.0.0 --port 8000 --reload

# API docs available at: http://localhost:8000/docs
# Health check: http://localhost:8000/api/health
```

Step 6: Start the Next.js Frontend

```
cd frontend
npm install
cp .env.example .env.local
# Set NEXT_PUBLIC_API_URL=http://localhost:8000

npm run dev

# UI available at: http://localhost:3000
# Admin panel at: http://localhost:3000/admin
```

Step 7: Run Evaluation (Optional)

```
cd eval
python run_eval.py --dataset ground_truth.json
# Expected output: 25/25 execution accuracy (100%)
```

11.3 Environment Variables Reference

Variable	Description
OPENAI_API_KEY	OpenAI API key with access to gpt-4o and text-embedding-3-small
DATABASE_URL	PostgreSQL connection string: postgresql://user:pass@host:port/dbname
QDRANT_URL	Qdrant service URL (default: http://localhost:6333)
POOL_SIZE	PostgreSQL connection pool size (default: 10)
MAX_ROWS_DEFAULT	Default row limit for queries if not set in guardrail_config (default: 500)
GPT_MODEL	OpenAI model to use (default: gpt-4o)
NEXT_PUBLIC_API_URL	Frontend: URL of the FastAPI backend (default: http://localhost:8000)